

TCP IP Communication in Action

What is TCP/IP?

In simple terms, TCP/IP is how devices communicate and transfer packets over the Internet.

Transmission Control Protocol (TCP) and Internet Protocol (IP) are two separate protocols that work together to ensure data is delivered to its intended destination within a network. IP obtains and defines the address; the IP address of the application or device the data must be sent to. TCP on the other hand is responsible for transporting data and ensuring it gets delivered to the destination application or device that IP has defined. To ensure the integrity of data transmitted between devices TCP/IP model was developed.

What is the TCP/IP model?

The TCP/IP model (Transmission Control Protocol/Internet Protocol) is a layered Internet and network communication framework that explains how data is communicated between devices over a network using standardized protocols to ensure reliable and efficient transmission.

Defined as a four-layer architecture consisting of Application, Transport, Internet, and Network Access layers.

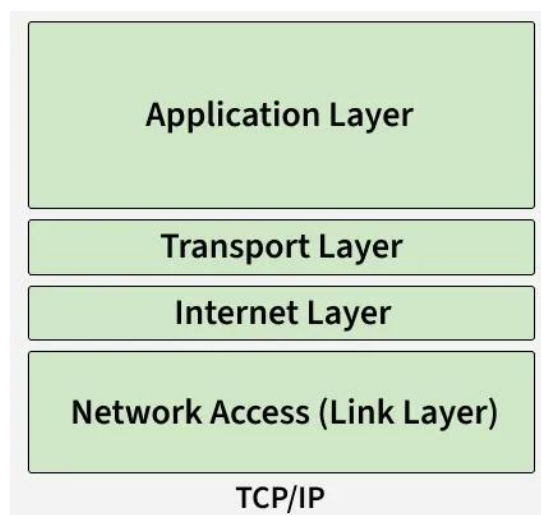


Fig. TCP/IP Model

Layers of TCP/IP Model

1. Application Layer

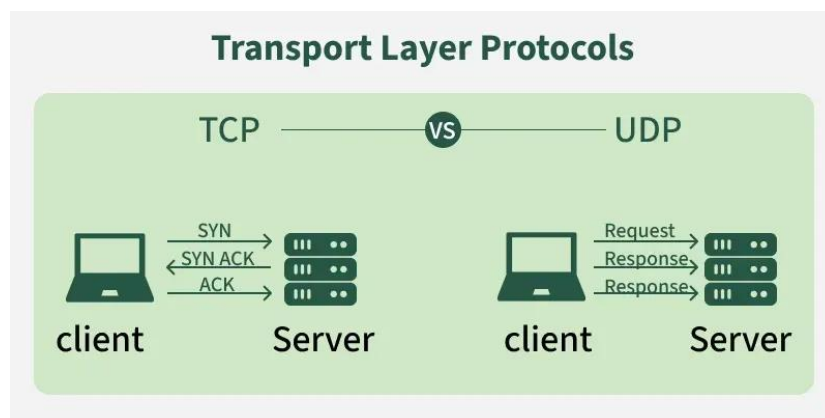
This is the top layer of the TCP/IP model, where applications like web browsers, email clients, and file-sharing tools interact with the network.

- Acts as a bridge between user applications and lower network layers.
- Supports protocols such as HTTP, HTTPS, FTP, SMTP, and DNS.
- Handles data formatting so information is correctly understood by both sender and receiver.
- Provides encryption for secure communication.
- Manages sessions to track ongoing connections.

2. Transport Layer

Ensures reliable and efficient delivery of data between devices, managing segmentation, ordering, and retransmission as needed.

- Breaks large messages into packets and reassembles them at the destination.
- TCP checks for errors, resends lost data, and ensures correct order.
- UDP provides low-latency, connectionless delivery without error checking.
- Prevents the receiver from being overwhelmed by regulating data flow.
- Uses port numbers to allow multiple applications to share the network simultaneously.



TCP (Transmission Control Protocol): TCP is used when reliability and accuracy are important. It ensures that data is delivered exactly as sent.

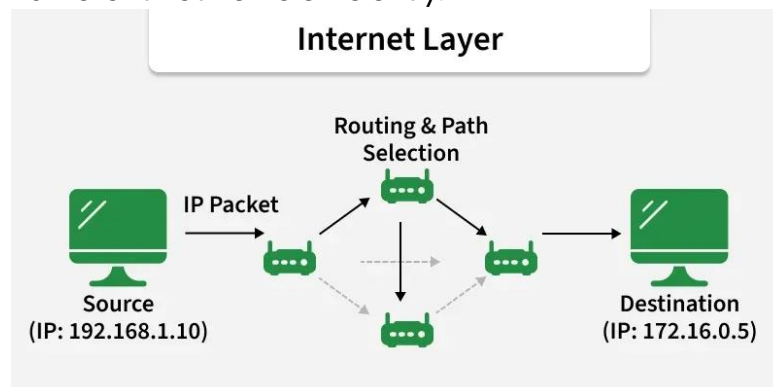
- TCP detects errors in the data using checksums to ensure integrity.
- If any data is lost or corrupted during transmission, TCP automatically resends it.
- Data is broken into packets, and TCP ensures these packets arrive in the correct sequence.
- TCP establishes a connection between sender and receiver before sending data, maintaining a stable session throughout the communication.
- Loading web pages, downloading files, sending emails, or any application where data must be complete and accurate.

UDP (User Datagram Protocol): UDP is used when speed is more important than perfect accuracy. It is faster but does not guarantee reliable delivery.

- UDP detects errors via checksums but does not verify or recover from them.
- Lost or damaged data is not resent.
- Packets may arrive out of order, and the protocol does not fix it.
- Because it avoids extra checks and connections, UDP is faster and uses fewer resources.
- Live video streaming, online gaming, VoIP calls, or real-time applications where speed matters more than reliability.

3. Internet Layer

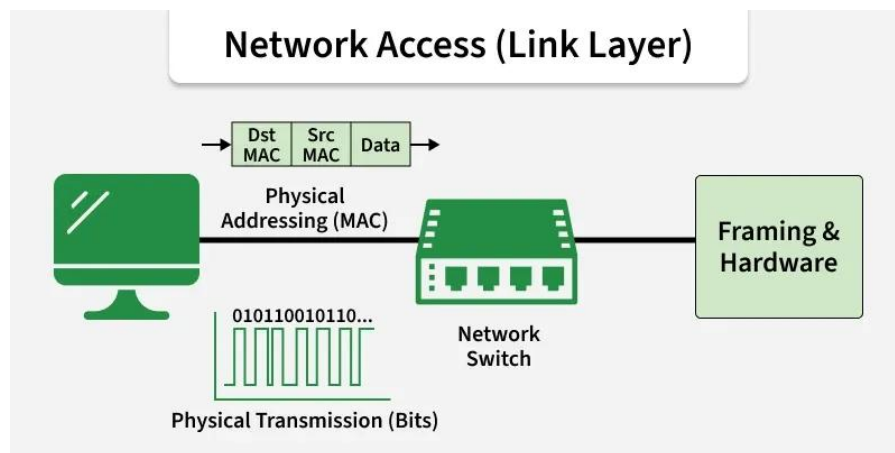
Responsible for addressing, packaging, and routing data packets so they can travel across networks and reach the correct destination device. It ensures that data can move between different networks efficiently.



- Assigns IP addresses to identify source and destination devices.
- Determines the best path for data to travel across networks.
- Breaks large packets into smaller ones for transmission and reassembles them at the destination.
- Primarily uses IP (Internet Protocol), along with supporting protocols like ICMP for error reporting and ARP for address resolution.

4. Network Access (Link Layer)

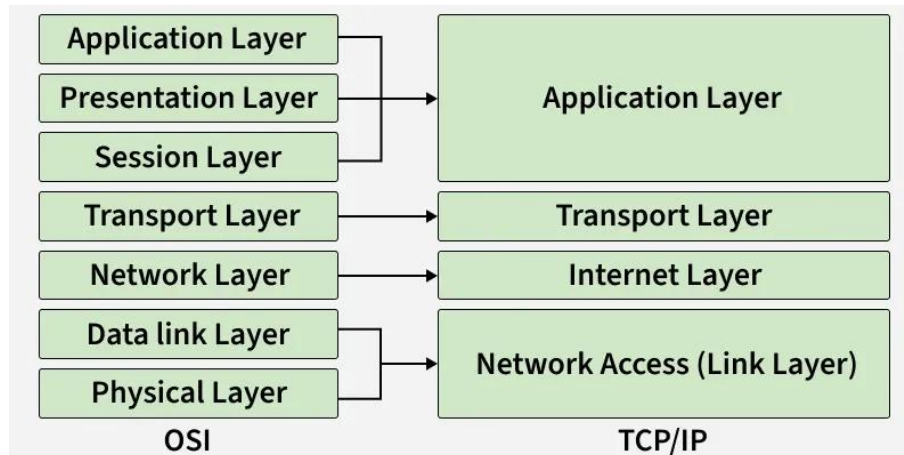
Responsible for physically transmitting data over network hardware, including cables, switches, and wireless connections. It handles how data is formatted for the network medium and ensures it reaches the next device on the path.



- Sends and receives raw bits over physical media like Ethernet cables, fiber optics, or Wi-Fi.
- Organizes data into frames for proper transmission and recognition by devices.
- Detects transmission errors using checksums or CRC.
- Uses hardware addresses to identify devices within the same network segment.
- Determines how multiple devices share the same physical medium, avoiding collisions.

Relationship between OSI and TCP/IP Model

Unlike OSI, which is mostly theoretical, TCP/IP is protocol-driven and focuses on actual communication needs. TCP/IP does not directly correspond to this model. TCP/IP combines several OSI layers into a single layer.



OSI Ref. Layer No.	OSI Layer Equivalent	TCP/IP Layer	TCP/IP Protocol Examples
5,6,7	Application, session, presentation	Application	HTTPS, NFS, NIS, DNS, LDAP, telnet, ftp, rlogin, rsh, rcp, RIP, RDISC, SNMP, and others
4	Transport	Transport	TCP, UDP, SCTP
3	Network	Internet	IP (IPv4, IPv6), ARP, ICMP
2	Data link	Network Access (Link)	PPP, IEEE 802.2
1	Physical	Network Access (Link)	Ethernet (IEEE 802.3), Token Ring, RS-232, FDDI, and others

Distinct Roles in the Protocol Stack

TCP (Transport Layer): Operates at Layer 4 of the TCP/IP model. It manages end-to-end communication between software applications. TCP is responsible for breaking large messages into smaller segments, numbering them, controlling transmission

speed, checking for errors, and requesting retransmissions for dropped data. It also uses port numbers to direct data to the correct application on a device.

IP (Internet Layer): Operates at Layer 3 of the TCP/IP model. It assigns unique numerical IP addresses to devices on a network and packages segments into datagrams or packets with address headers, guiding them along the path across routers and interconnected networks.

How They Work Together to Deliver Data

1. Establishing a Connection (Three-Way Handshake): Before transmitting data, TCP establishes a connection-oriented session between the sender and receiver through a three-step handshake process, synchronizing both devices.

2. Segmentation and Encapsulation (Sender Side):

- The sending application passes message data down to TCP at the Transport Layer.
- TCP breaks the message into smaller segments, attaches a header containing sequence numbers and port numbers, and passes them down to IP.
- IP encapsulates each TCP segment into an IP packet, tagging it with source and destination IP addresses.

3. Routing Across Networks:

- IP routes each packet across various network devices (such as routers and switches) toward the destination IP address.
- Individual packets can travel independently and take different physical routes to avoid network congestion or unavailable links.

4. Reassembly and Error Recovery (Receiver Side):

- Upon arrival, the receiving device's IP layer extracts the packets, verifies the destination IP address, and passes the payloads up to TCP.
- TCP inspects the segments for errors (using checksums), discards duplicate packets, and uses sequence numbers to reassemble the data into its original order.
- If any packets were lost or corrupted in transit, TCP automatically requests retransmission from the sender.

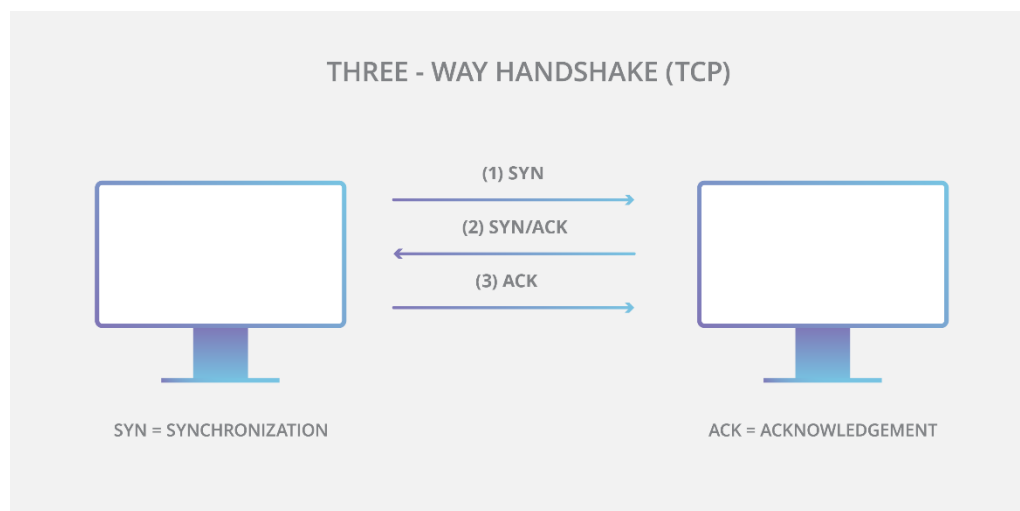
- Once TCP confirms the message is complete and accurately restored, it delivers the final data to the destination application.

Summary Analogy

A helpful way to view their relationship is that **IP acts as the address and location finder** (identifying where the device is located on a network), while **TCP acts as the reliable courier service** (packaging items into numbered boxes, verifying receipt, correcting errors, and reassembling the contents at the destination¹).

TCP three-way handshake

The **TCP three-way handshake** (or 3-step handshake) is the process TCP uses to establish a reliable, connection-oriented session between a client device and a server before any data transmission occurs. This process ensures that both the sender and receiver are synchronized and ready to exchange packets concurrently.



1. SYN — Connection Request (Device to Server):

The **SYN** (short for **Synchronize**) flag is one of the essential control bits located in the **TCP segment header**. Its primary purpose is to **initiate a TCP connection** and synchronize initial sequence numbers between two endpoints before data transfer begins.

During the TCP three-way handshake, the **SYN** flag is used in the first two steps to negotiate the session parameters:

1. **SYN (Step 1 — Client Request):** The initiating device (client) sends a packet with the **SYN flag set to 1**. This segment contains the client's randomly generated

Initial Sequence Number (ISN) (e.g., Seq = 1000). It signals to the server: *"I want to start a connection, and this is where my numbering begins."*

2. **SYN-ACK (Step 2 — Server Response):** The receiving server accepts the request and replies with a packet that has both the **SYN and ACK flags set to 1**. It acknowledges the client's initial sequence number (Ack = 1001) and sends its own starting sequence number (Seq = 5000).
3. **ACK (Step 3 — Connection Confirmation):** The client sends a final packet with the **ACK flag set to 1** (Ack = 5001), confirming the receipt of the server's sequence number. At this point, the connection is established and data can flow in both directions.

2. SYN-ACK — Synchronization & Acknowledgment (Server to Device):

SYN-ACK (short for **Synchronize-Acknowledge**) is the packet flag combination used in **Step 2 of the TCP three-way handshake**. It serves as the server's response to an incoming connection request.

1. Dual Function in Step 2:

- **Acknowledgment (ACK):** The server confirms and acknowledges receipt of the client's initial connection request (SYN).
- **Synchronization (SYN):** The server sends its own initial sequence number back to the client to establish the data channel from the server to the device.

2. Enabling Two-Way Communication:

- Because TCP is connection-oriented and bidirectional, both the client and server must synchronize their sequence numbers and acknowledge each other before transferring data.
- The SYN-ACK packet bridges the initial request and establishes the server's side of the connection.

3. Completing the Handshake:

- Once the client receives the SYN-ACK packet, it responds with a final **ACK** (Step 3).
- After this step is complete, the TCP connection is fully open, allowing information to flow in both directions simultaneously.

3. ACK — Confirmation & Session Opening (Device to Server):

And with the **ACK**, **TCP three-way handshake** is complete.

The **ACK (Acknowledgment)** flag plays a critical role in session creation, ongoing data transfer, and session termination.

1. Completing the Three-Way Handshake

In **Step 3** of connection establishment, the client sends an **ACK** segment back to the server in response to its SYN-ACK. This confirms that both endpoints have synchronized sequence numbers and agreed on connection parameters, officially opening the full-duplex socket session for application data.

2. Ensuring Reliable Ongoing Data Transfer

- **Delivery Verification:** Once a session is active, almost every TCP segment sent includes an active ACK flag along with an acknowledgment sequence number. This verifies to the sender that incoming packets arrived safely without corruption.
- **Triggering Retransmissions:** If an expected ACK is missing or delayed beyond a retransmission timeout, the sending transport layer assumes the packet was lost or dropped and automatically resends it.
- **Flow Control:** ACK messages communicate the receiver's available buffer space (window size), allowing TCP to dynamically throttle or increase transmission rates to prevent network congestion or receiver buffer overflow.

3. Graceful Teardown

- During the **4-step teardown process**, ACK packets confirm receipt of FIN (Finish) requests from both client and server, ensuring both sides have completed their data transfers before releasing system resources.

Port Numbers and Socket Pairs

While **IP addresses** identify physical or virtual devices on a network, they are not enough on their own to know *which application* on a computer should receive incoming data packets.

Port Numbers (Application Addressing)

To solve this, the Transport Layer uses **port numbers** (also called application addressing). Port numbers act as specific digital doorways or endpoints for applications¹:

- **Well-known ports** (e.g., port 80 for HTTP, port 443 for HTTPS) identify standard network services.
- Port numbers allow multiple applications (such as a web browser, email client, and streaming app) to share the same network interface simultaneously without data streams mixing together.

Socket Pairs (Identifying Unique Connection Streams)

In networking, a **socket** is the combination of an **IP address and a port number** (e.g., 192.168.1.10:54321).

To uniquely identify a specific connection stream across an internetwork, TCP uses a **socket pair**—a 4-tuple consisting of four specific pieces of information:

1. **Source IP Address** (Sender's device location)
2. **Source Port Number** (Sender's specific application process)
3. **Destination IP Address** (Receiver's device location)
4. **Destination Port Number** (Receiver's specific application service)

By tracking this unique 4-tuple header parameter, the receiving operating system can separate and direct concurrent socket connections directly to the exact application process meant to handle them, even if hundreds of separate browser tabs are connected to the same web server at the same time.

At the **Transport Layer** (Layer 4) of the TCP/IP suite, **port numbers** provide application addressing. While IP addresses direct packets to the correct physical or logical host on a network, port numbers ensure that incoming data is routed to the specific application or software process running on that host.

1. Source Port vs. Destination Port

- **Source Port:**

- Identifies the specific application or process on the **originating (sender)** host.
- When a client (like a web browser) initiates a connection, its operating system dynamically assigns an **ephemeral (temporary) source port**.
- It acts as a return doorway so the receiving server knows where to direct incoming reply packets.

- **Destination Port:**

- Identifies the target service or process on the **receiving host**.
- Servers listen on standardized **well-known ports** so clients know where to reach them (for example, Port **443** for HTTPS, Port **80** for HTTP, or Port **21** for FTP).

Common protocols and their ports

Protocol	Primary Function	Default Port(s)
HTTP	Web page access	80
HTTPS	Secure web browsing	443
FTP	File transfer	20, 21
SMTP	Sending emails	25, 587
POP3	Downloading emails	110
IMAP	Syncing emails	143, 993
DNS	Domain name resolution	53
DHCP	IP address assignment	67, 68
SSH	Secure remote access	22
Telnet	Insecure remote access	23
SNMP	Network management	161, 162

NFS	Remote file sharing	2049
-----	---------------------	------

Why HTTPS normally uses TCP port 443?

HTTPS relies on **TCP Port 443** to deliver secure web content reliably across networks. This specific combination of protocol and port number is used for several key reasons:

1. Why TCP?

- **Reliability and Integrity:** HTTPS carries encrypted web page data, scripts, and media that must arrive completely error-free and in exact sequence.
- **Connection Management:** TCP establishes a reliable connection session (via the three-way handshake) and handles packet reassembly, error checking, and retransmissions. This stable transport layer is required before the Transport Layer Security (TLS/SSL) protocol can negotiate encryption keys and secure the web session.

2. Why Port 443?

- **Application Addressing:** In the TCP/IP model, port numbers act as transport-layer endpoints that direct incoming data packets to the specific application or service meant to process them on a server.
- **Separation from Unencrypted Traffic (Port 80):** Standard unencrypted HTTP traffic uses **Port 80** by default. Assigning **Port 443** specifically for HTTPS allows web servers to separate plain text HTTP traffic from encrypted TLS/SSL traffic seamlessly without port conflicts.
- **Firewall and Network Policy Management:** Dedicated, standardized port numbers allow firewalls, proxies, and network security gateways to recognize encrypted web traffic easily, applying distinct security, routing, and filtering policies to Port 443 without interfering with standard web browsing on Port 80.

Ephemeral client port

An **ephemeral client port** (also called a dynamic or temporary port) is a short-lived port automatically assigned by a client device's operating system when initiating an outbound connection to a server.

1. How Ephemeral Ports Function

- **Fixed Servers vs. Dynamic Clients:** Web servers and network services listen on standardized, **well-known ports** (such as Port 443 for HTTPS or Port 80 for HTTP) so clients know where to send requests¹. Client applications, however, do not need a fixed port.
- **Automatic Allocation:** When an application (such as a web browser) makes a network request, the operating system selects an unused, high-numbered port from its dynamic pool (typically in the range 49152–65535, though ranges vary by OS) to serve as the client's temporary endpoint.

2. Why Ephemeral Ports Are Necessary

1. **Return Address Routing:** The client port provides a specific transport-layer doorway so the receiving server (and the local operating system) knows exactly which application process on the client device should receive the returning data packets.
2. **Completing the Socket Pair:** To track concurrent connections, TCP identifies each unique stream using a 4-tuple **socket pair**: (Source IP, Ephemeral Source Port, Destination IP, Destination Port)

By assigning a distinct ephemeral port to each outbound request, your device can open hundreds of simultaneous browser tabs to the exact same web server without streams mixing together.

3. Lifecycle of an Ephemeral Port

- **Allocation:** Assigned automatically when the client sends the initial SYN packet to establish a connection.
- **Active Exchange:** Used throughout the TCP session to route data segments to the correct socket.
- **Release & Reuse:** Once the session completes its 4-step teardown process (FIN/ACK) and exits its brief waiting state, the OS releases the port back into the dynamic pool to be recycled for future connections.

TCP Reliability

TCP (Transmission Control Protocol) provides reliable, end-to-end data delivery across unreliable network infrastructures. It achieves this reliability through a suite of core mechanisms:

1. Connection-Oriented Session Setup

- **Three-Way Handshake:** TCP establishes a stable session between the sender and receiver before transmitting application payload. This ensures both endpoints are active and synchronized prior to data exchange.

2. Segmentation and Sequence Numbering

- **Segmentation:** Large messages are broken down into smaller packets or segments for manageable transmission.
- **Sequence Numbers:** Each segment is tagged with a sequence number, allowing the receiving host to discard duplicate data and reassemble packets back into their exact original order.

3. Acknowledgments (ACKs) and Retransmissions

- **Positive Acknowledgments:** Receiving devices return acknowledgment (ACK) signals upon successfully receiving packets.
- **Automatic Retransmission:** If a packet is lost, dropped, or corrupted, or if an acknowledgment fails to return within an expected time frame, TCP automatically retransmits the missing data.

4. Error Checking

- **Checksum Verification:** TCP attaches checksums to segment headers to detect bit corruption. If a segment fails the checksum check on arrival, it is discarded and re-requested.

5. Flow and Congestion Control

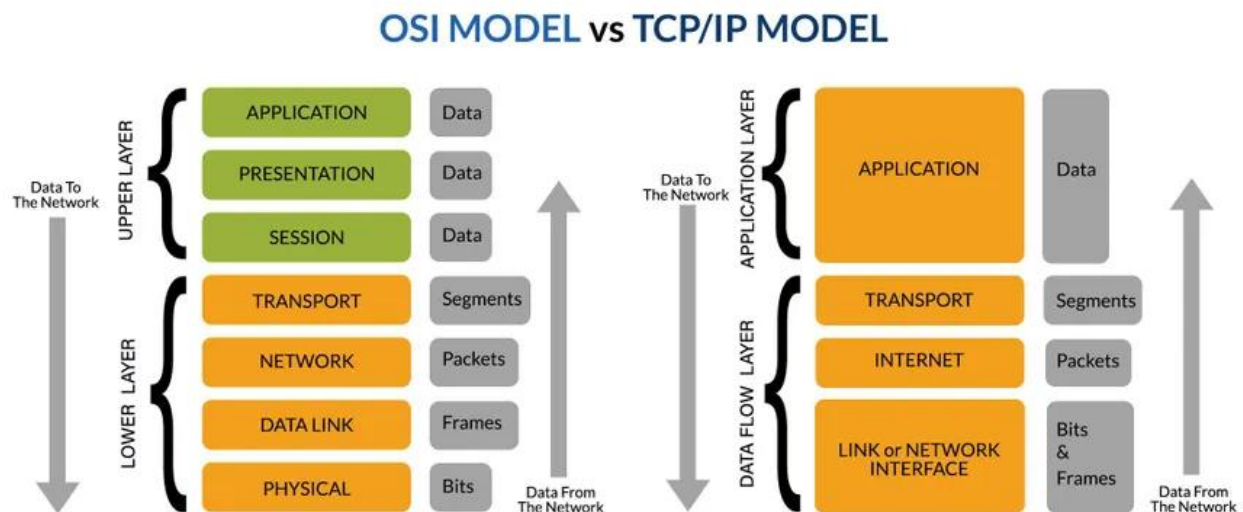
- **Flow Control:** TCP regulates the rate of data transmission to match the processing speed of the destination host, preventing receiver buffer overruns.
- **Congestion Control & Windowing:** Using mechanisms like dynamic windowing, TCP adjusts its transmission speed based on detected network bottlenecks and congestion to maintain session stability.

Comparison between TCP and UDP

Feature	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection Type	Connection-oriented (requires handshake)	Connectionless (no handshake)
Delivery Guarantee	Guaranteed delivery & automatic retransmission	Unreliable / non-guaranteed (lost packets are dropped)
Packet Ordering	Strict sequential reassembly	Packets may arrive out of order
Speed & Overhead	Slower with higher header and state overhead	Fast with minimal latency and header overhead
Flow & Error Control	Yes (manages buffers, sequence IDs, retransmissions)	Basic error detection only (no recovery or flow regulation)
Common Uses	Web browsing (HTTP/HTTPS), file transfers (FTP), email (SMTP), SSH	Live video streaming, online gaming, VoIP, DNS queries, TFTP

Data Journey across TCP/IP Model Layers

Mapping TCP/IP to the OSI Reference Model



As data travels across a network, it undergoes a transformation process known as **encapsulation** on the sending host and **decapsulation** on the receiving host.

1. Data Transformation Across Layers (Encapsulation & Decapsulation)

Sender Side: Encapsulation (Top-Down)

When an application (such as a web browser opening an HTTPS website) initiates communication, data moves down the TCP/IP stack, where each layer wraps the payload with its own control headers:

1. Application Layer (Layer 4):

- **Action:** User software creates the initial message or payload (e.g., an HTTP/HTTPS web request).
- **Data Unit:** Data / Payload.

2. Transport Layer (Layer 3):

- **Action:** TCP accepts the application data and breaks large messages into smaller chunks. It attaches a **TCP Header** containing source and destination port numbers (e.g., Port 443 for HTTPS), sequence numbers, and error-checking checksums.
- **Data Unit:** Segment (or Datagram if using UDP).

3. Internet Layer (Layer 2):

- **Action:** IP encapsulates the TCP segment inside an **IP Header**, which contains logical source and destination IP addresses along with routing parameters.
- **Data Unit: Packet** (or IP Datagram).

4. Network Access / Link Layer (Layer 1):

- **Action:** The Link layer wraps the IP packet with a **Frame Header** (containing hardware MAC addresses) and a **Frame Trailer** (containing error detection like Cyclic Redundancy Check / CRC).
- **Data Unit: Frame.**
- **Physical Transmission:** Network hardware (NICs, cables, Wi-Fi) converts frames into raw **Bits** (electrical impulses, light pulses, or radio signals) to cross physical media.

Receiver Side: Decapsulation (Bottom-Up)

When the destination server receives the physical signal, the process runs in exact reverse:

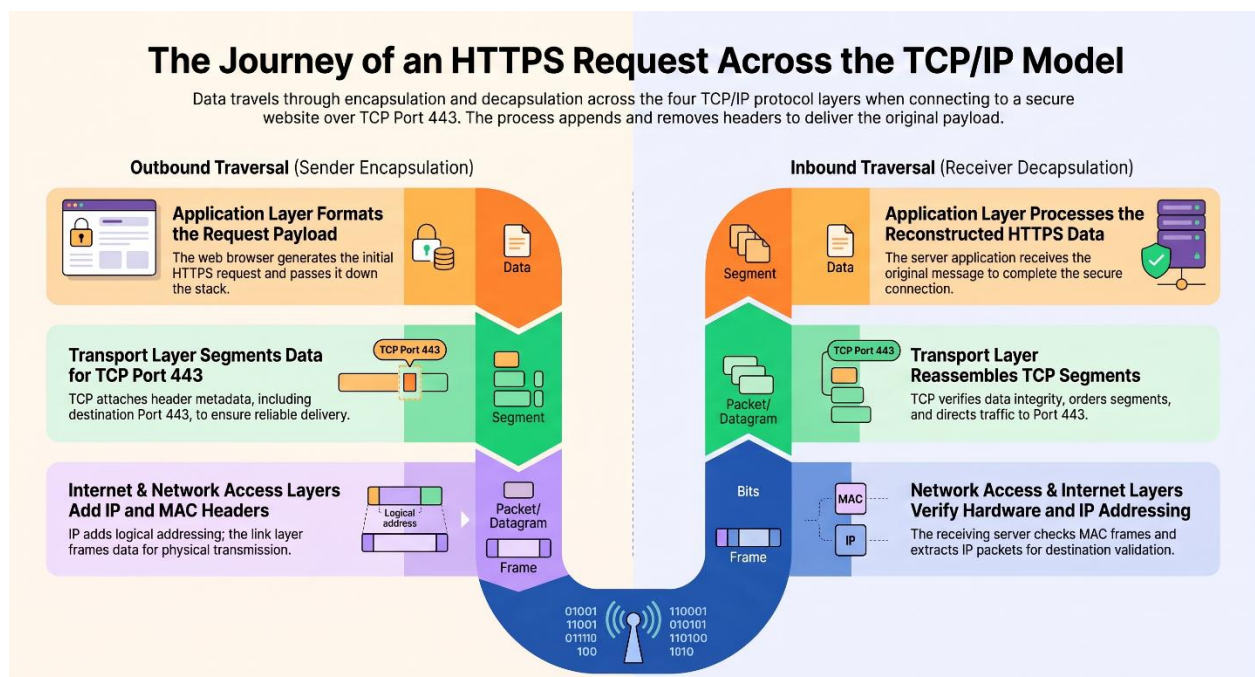
- **Network Access Layer:** Receives raw bits, verifies frame integrity using the CRC checksum, strips the frame header and trailer, and extracts the packet.
- **Internet Layer:** Checks the destination IP address, verifies routing, strips the IP header, and passes the segment up to the Transport layer.
- **Transport Layer:** Inspects TCP sequence numbers and port numbers, reassembles data segments, verifies checksums, sends acknowledgment (ACK) signals, strips the TCP header, and hands the payload up to the destination process.
- **Application Layer:** Receives the fully reassembled, un-encapsulated data stream for final software processing.

2. Diagram Breakdown: Protocol Data Units (PDUs)

The encapsulation process is best depicted through its **Protocol Data Units (PDUs)** at each layer:

TCP/IP Layer	Added Header	Protocol Data Unit (PDU) Name	Primary Identifiers
Application Layer	Application Data / Encryption	Data / Message	Application Protocols (HTTP, TLS, DNS)
Transport Layer	TCP Header	Segment	Source & Destination Ports (e.g., Port 443)
Internet Layer	IP Header	Packet / Datagram	Source & Destination IP Addresses
Network Access Layer	Frame Header & Trailer (CRC)	Frame	Source & Destination MAC Addresses
Physical Medium	Signals / Media Encoding	Bits	Raw binary signals (0s and 1s)

HTTPS Data Journey across TCP/IP Model Layers



References

What is TCP/IP? (n.d.-b). <https://www.cloudflare.com/learning/ddos/glossary/tcp-ip/>

TCP/IP in action. Visualize TCP/IP in network traffic... | by hira hussain | medium. (n.d.-b). <https://hiraalihussain88.medium.com/tcp-ip-in-action-85ae16e966b1>

GeeksforGeeks. (2026, June 11). TCP/IP model. <https://www.geeksforgeeks.org/computer-networks/tcp-ip-model/>

Introducing the TCP/IP Protocol Suite. Introducing the TCP/IP protocol suite - system administration guide: IP services. (2011, August 1). https://docs.oracle.com/cd/E18752_01/html/816-4554/ipov-6.html

Rev. (2024, May 27). *The journey of a network packet: A TCP/IP guide*. DEV Community. <https://dev.to/3rev/demystifying-tcpip-understanding-the-journey-of-a-network-packet-38g7>

Moore, S. (2026, April 3). *TCP/IP model explained: Layers, protocols, and best practices*. Exabeam. <https://www.exabeam.com/explainers/osi-layers/tcp-ip-model-explained-layers-protocols-and-best-practices/>

What is TCP/IP and how does it work? Erie Institute of Technology. (2024, August 9). <https://erieit.edu/what-is-tcp-ip-how-does-it-work/>